Design, Validation and Verification document

1.0 Title of project and team members

Project Title: APEX (AI Patient Experience Simulator)
Team Members:

1. Tung Ho - 400370087
2. Asher Haroon - 400411801
3. Michael Fedotov - 400384934
4. Hammad Ur Rehman - 400370677
5. Christian Canlas - 400386605
6. Aaryan Kandwal - 400368334
7. Samir Matani - 400371096

2.0 Revision History

Version Number	Authors	Description	Date
0	● Tung Ho ● Asher Haroon ● Michael Fedotov ● Hammad Ur Rehman ● Christian Canlas ● Aaryan Kandwal ● Samir Matani	First Draft	January 14th, 2026
1	● Tung Ho ● Asher Haroon ● Michael Fedotov ● Hammad Ur Rehman ● Christian Canlas ● Aaryan Kandwal ● Samir Matani	Final Version	April 5th, 2026

3.0 Introduction

3.1 Project Purpose

Effective clinical communication and empathy are critical skills for medical professionals, particularly when delivering difficult news to patients. However, opportunities for structured, repeatable, and scalable empathy training are limited due to resource constraints and variability in human role-play scenarios.

APEX (AI Patient Experience Simulator) is an AI-powered empathy training platform designed to allow medical trainees to practice patient-centered communication through simulated clinical conversations. The system leverages large language models (LLMs) to act as virtual patients and provides structured post-session feedback based on the SPIKES protocol. This enables trainees to receive immediate, consistent, and repeatable feedback on their empathetic communication skills.

3.2 Document Purpose

The purpose of this document is to provide a detailed description of the design and architecture of the APEX system. It outlines the system components, their interactions, and their relationship to the project requirements as defined in the Software Requirements Specification (SRS). This document also identifies potential undesired behaviours for each component to support future verification and validation efforts.

3.3 Project Scope

The APEX system consists of a web-based application that allows users to:

- Authenticate and manage training sessions
- Select predefined clinical cases
- Engage in text-based (and optional voice-based) conversations with simulated patients
- Receive structured feedback aligned with the SPIKES protocol
- View session transcripts and empathy-related metrics

The system is intended for training and educational purposes only and does not provide medical advice or diagnostic recommendations.

3.4 Assumptions and Invariants
3.4.1 Assumptions

- Users are medical trainees or instructors with basic clinical communication knowledge
- Users have access to a stable internet connection
- AI-generated responses are probabilistic and may vary between sessions
- The system is used in a controlled educational setting

3.4.2 Invariants

- The system does not replace real clinical training or supervision
- No real patient data is used within the system
- All final clinical judgments remain the responsibility of human practitioners
- Stored session data remains immutable once a session is closed

---

4.0 Project Component Diagram (High-Level)

The project component diagram is illustrated in Appendix A, Figure A.5.

The APEX system is composed of interconnected frontend, backend, and external components that support user interaction, session management, AI-driven dialogue, and analytics.

1. The Frontend UI sends user actions and session requests to the Backend.
2. The Backend returns processed responses and session updates to the Frontend UI.
3. The Backend sends prompts and contextual data to External LLM Services for AI-driven dialogue and analysis.
4. External LLM Services return generated responses and structured outputs to the Backend.
5. The Backend stores session data, dialogue turns, and feedback in the Database.

4.1 Backend Component Diagram

The project component diagram is illustrated in Appendix A, Figure A.6.

1. The frontend sends REST or WebSocket requests to the backend's API controllers.
2. The API controllers route each request to the appropriate backend service based on its purpose, such as authentication, case retrieval, session management, or dialogue processing.
3. For conversational input, the dialogue service loads and updates session context and protocol state as needed.
4. The dialogue service performs local natural language understanding by running internal rule-based analysis and span detection on the user's input.
5. When a simulated patient response is required, the dialogue service invokes external language model services through the backend's adapter layer and receives the generated output.
6. After processing a turn, the dialogue service calls the scoring service to compute empathy metrics, question classifications, and other evaluation signals.
7. The scoring service may use both internal language analysis and external language model services to produce structured feedback.
8. Conversation turns, session updates, and computed feedback are persisted through the data repositories to the database.
9. Once processing is complete, the API controllers return dialogue responses, session updates, and feedback results back to the frontend.
10. If voice interaction is enabled, the dialogue service may optionally invoke speech processing adapters before continuing the same processing flow.

5.0 Relationship Between Project Components and Requirements

System Component	Requirements Covered

Frontend UI	P0: Interactive training interface, session control Nonfunctional: Usability, accessibility
Backend API	P0: Session persistence and authentication P1: Role-based access P0: Simulated patient interaction P1: SPIKES stage detection P0: Empathy feedback generation
Database	P0: Transcript and feedback storage Nonfunctional: Reliability
External Services	P0: Reliable patient response generation for simulated dialogue based on case context and session state

---

6.0 Project Components

6.1 Frontend UI

6.1.1 Normal Behaviour

The UI allows users to log in, select a clinical case, initiate a training session, and interact with a simulated patient through a chat interface. Upon completion of a session, users can view structured feedback and session summaries.

6.1.2 API and Structure

The UI communicates with the backend via RESTful API endpoints to:

- Authenticate users
- Create and manage sessions
- Submit dialogue turns
- Retrieve feedback and session summaries

Data is exchanged in JSON format.

6.1.3 Implementation

The UI is implemented using React with TypeScript. State management is handled through a centralized store, and routing is managed via a client-side router. The chat interface renders conversation turns dynamically and updates in real time as responses are received from the backend.

6.1.4 Potential Undesired Behaviours

- User input submission failures due to network interruptions
- UI latency when backend responses are delayed
- Incomplete rendering of session feedback
- Unauthorized access if authentication tokens are mismanaged

6.2 Backend

6.2.3 API LAYER (Controllers)

Cases Controller

a. Normal behaviour: provides CRUD operations for patient simulation cases with role-based access control.

b. API (inputs/outputs): inputs include case metadata (title, script, objectives, difficulty, category) and query parameters for filtering and pagination. Outputs include case objects and paginated lists of cases.

c. Implementation

- Endpoints:
    - `GET /cases, POST /cases, GET /cases/{id}, PATCH /cases/{id}, and DELETE /cases/{id}`
- An instructor role is required to create/update/delete
- Delegates logic to `CaseService`

d. Potential undesired behaviours: missing input sanitization, no validation on difficulty/category enums, and no content length limits.

Sessions Controller

a. Normal behaviour: manages session creation, turn processing (text/audio), and session closure.

b. API (inputs/outputs)

- Inputs: session creation parameters (case ID), text turns, and audio files.
- Outputs: session state, patient responses, and feedback metrics.

c. Implementation

- Endpoints:
    - `POST /sessions, POST /sessions/{id}/turns, POST /sessions/{id}/audio, and POST /sessions/{id}/close`
- Instantiates adapters (LLM, Whisper, NLU)
- Orchestrates `DialogueService → ScoringService`

d. Potential undesired behaviours

- Adapter instantiation per request (performance overhead)
- No audio size or format validation
- Race conditions on concurrent turn submissions

6.3.2 BUSINESS LOGIC LAYER (Services)

Case Service

a. Normal behaviour: manages creation, retrieval, updating, and deletion of cases.

b. API (inputs/outputs)

- Inputs: case metadata, filters, and pagination parameters.
- Outputs: case entities and lists.

c. Implementation

- Uses `CaseRepository`
- Validates case existence
- Partial updates via `model_dump(exclude_unset=True)`

d. Potential undesired behaviours: duplicate cases not detected, and enum values not validated.

Session Service

a. Normal behaviour: controls session lifecycle and session metadata.

b. API (inputs/outputs): inputs include Case ID, user ID, session ID. Outputs include session entity with turn history.

c. Implementation

- Initializes sessions at SPIKES stage “Setting”
- Calculates session duration on closure
- Retrieves turns via `TurnRepository`

d. Potential undesired behaviours: missing permission checks, unlimited concurrent sessions, and orphaned sessions remain open.

Dialogue Service (SPIKES Engine)

a. Normal behaviour: processes dialogue turns and advances SPIKES stages.

b. API (inputs/outputs): inputs include session ID, clinician turn text. Outputs include patient response turn, and metrics.

c. Implementation

- Pipeline: input → NLU → turn creation → LLM → EO detection → SPIKES update
- NLU via `SimpleRuleNLU`
- Span detection with character offsets
- LLM generates patient responses
- SPIKES stage progression via keyword and fallback logic

d. Potential undesired behaviours: no validation of LLM output length/content

Scoring Service

a. Normal behaviour: generates empathy and SPIKES feedback using AFCE metrics.

b. API (inputs/outputs): inputs include the session ID. Outputs include scores, metrics, and structured feedback.

c. Implementation

- Empathy score (EO coverage, dimension match, timing)
- SPIKES score (coverage, sequence, empathy during E)
- Links EOs to responses within two turns
- Computes missed opportunities

d. Potential undesired behaviours: serialization errors with special characters and no caching (recalculation on every call)

6.3.3 NLU COMPONENTS

Rule-Based NLU (SimpleRuleNLU)

a. Normal behaviour: detects empathy cues, question types, tone, and AFCE constructs.

b. API (inputs/outputs)

- Inputs: text string
- Outputs: classified cues, tone, spans

c. Implementation: keyword-based empathy detection, question classification (open vs closed), tone analysis, and delegates span extraction to `SpanDetecto.`

d. Potential undesired behaviours: no negation handling and English-only support

Span Detector

a. Normal behaviour: identifies AFCE and SPIKES spans with offsets.

b. API (inputs/outputs)

- Inputs: text string
- Outputs: span objects (start, end, confidence, type)

c. Implementation

- EO detection across dimensions and types, elicitation and response detection, SPIKES stage keyword matching, and overlap resolution by confidence.

d. Potential undesired behaviours

- Fragmented multi-word spans, imperfect overlap resolution, no contextual understanding, and arbitrary confidence scores.

6.3.4 Data Access Layer (Repositories)

Repositories (User, Case, Session, Turn, Feedback)

a. Normal behaviour: provide CRUD access to persistent storage.

b. API (inputs/outputs): inputs include ORM entities and filters. Outputs include Database records.

c. Implementation (Methods): `get_by_id`, `get_all`, `create`, `update`, `delete`, SQLAlchemy ORM, and pagination via `offset` and `limit`

d. Potential undesired behaviours: no query timeouts, no soft deletes, and incomplete transaction rollback

Speech Adapter (Whisper)

a. Normal behaviour: Transcribes clinician audio input to text.

b. API (inputs/outputs): inputs include audio bytes (wav/mp3/ogg). Outputs include transcribed text.

c. Implementation: writes audio to a temporary file and calls Whisper API externally.

d. Potential undesired behaviours: missing audio validation and temporary files not deleted on failure.

6.5 Database

6.5.1 Normal Behaviour

The database provides persistent storage for the APEX system, including users, synthetic patient cases, dialogue sessions, turn-level interactions, and automated

feedback. Session data is written incrementally during conversations and finalized upon completion, supporting both real-time interaction and post-session analysis.

6.5.2 API and Structure

APEX uses a normalized PostgreSQL schema with foreign key relationships to maintain referential integrity across users, sessions, turns, and feedback records. Analytical outputs, including empathy metrics and SPIKES annotations, are stored in serialized form for flexibility. Database access is managed through the backend API with role-based access control and restricted write permissions.

6.5.3 Implementation

The database is implemented using PostgreSQL and hosted on Supabase. Application access is managed through SQLAlchemy ORM, providing transactional guarantees and automatic rollback on failure. Core tables are indexed to support efficient session retrieval and analytics.

6.5.4 Potential Undesired Behaviors

- Incomplete or corrupted writes, where sessions, turns, or feedback records are only partially persisted due to interrupted database transactions
- Unauthorized or unintended data access, caused by misconfigured authentication, role-based access control, or API permissions
- Performance degradation under load, particularly during high-frequency turn logging or analytics-heavy queries

---

7.0 User Interface

The APEX frontend consists of multiple views, including:

- Login screen, case selection dashboard, chat-based training interface, and Feedback summary page

The interface prioritizes clarity, minimal cognitive load, and ease of navigation to support effective learning. Visual indicators are used to display session progress and SPIKES stages.

The chat-based training interface optionally supports voice-based interaction through microphone input and audio playback, with automatic fallback to text-based interaction if audio is unavailable.

Screenshots which represent the APEX user interface, illustrating the login flow, case selection, chat-based training interface, and post-session feedback view, are

provided in Appendix B. These screenshots illustrate the flow of the user interface (UI) that matches the V0 implementation that is described in this document.

8.0 Validation and Verification

This section describes the validation and verification activities used to ensure that the system meets its functional and non-functional requirements. Verification focuses on confirming the correct behaviour of individual components, while validation evaluates performance, robustness, and usability under realistic operating conditions.

analytics(michael), audio(asher), case(christian), dialogue(asher), feedback(aaryan), scoring(aaryan), session(asher), plugin(christian), db(samir), admin(samir), research(michael)

8.1 Authentication

The authentication component manages user identity, session state, and access control across the backend and frontend. It supports Supabase JWT-based authentication, profile retrieval through authenticated endpoints, email existence checks during account workflows, frontend session initialization, protected route enforcement, and role-based authorization for restricted views.

8.1.1 Verification of Authentication

Verification focused on correct token handling, identity resolution, session state management, and route protection. Backend tests verified JWT decoding for valid tokens and confirmed rejection of expired, malformed, or incorrectly signed tokens. Role-scope checks were also verified to ensure that administrative permissions inherit lower-level access while unauthorized roles are denied.

API-level verification covered /v1/auth/email-exists and /v1/auth/me. Tests confirmed that email existence queries return correct results for missing and existing users, including case-insensitive email matching. Authenticated profile retrieval was verified for missing authorization headers, invalid token payloads, malformed subject identifiers, missing database users, and successful user resolution when the JWT subject matches a stored Supabase identity.

Frontend verification covered authentication state transitions and guarded navigation. Tests confirmed that session initialization correctly handles both empty and active sessions, that profile refresh populates user data when a token exists, and that logout clears client-side authentication state. Additional tests verified that protected routes show loading during initialization, redirect unauthenticated users to login, allow authenticated users through, and block users whose roles do not satisfy route requirements.

8.1.2 Validation of Authentication

Validation assessed whether the authentication workflow supports secure and usable access to the training platform. The combined backend and frontend tests show that authenticated users can establish a session, retrieve their profile, and access protected application areas without manual state repair, while invalid or expired credentials are rejected before protected resources are exposed.

The tests also validate that authentication behaves robustly in realistic application scenarios. Session restoration after reload, lazy profile synchronization, and automatic logout on authorization failures support consistent user experience while preserving security boundaries. Role-based route checks further validate that users are directed only to features appropriate for their permissions.

These results demonstrate that the authentication component provides reliable identity verification, resilient session handling, and effective access control for both general and role-restricted application flows.

8.2 Session

The session component manages an active training encounter between a user and a selected case. It is responsible for session creation, reuse of in-progress sessions, forced creation of new sessions, lifecycle transitions from active to completed, timestamp tracking, SPIKES-stage context, and freezing runtime configuration such as evaluator, patient model, and metrics plugins for consistent execution.

8.2.1 Verification of Session

Verification focused on correct session lifecycle behavior, persistence of session metadata, and enforcement of session constraints. Unit tests validated session creation for a given user and case, confirming that newly created sessions store the correct user ID, case ID, active state, and start timestamp. Repeated creation requests without force_new were verified to reuse the existing active session rather than creating duplicates, while requests with force_new correctly produced a new session record.

Session completion behavior was verified by closing an active session and confirming transition to the completed state with an end timestamp recorded in UTC. Additional tests verified timestamp serialization and ensured that session start and end times are emitted in a consistent format suitable for downstream APIs and clients.

Configuration persistence during session creation was also verified. Tests confirmed that evaluator selection can come from request parameters, application settings, or case-level overrides, and that the chosen evaluator name and version are frozen onto the session. Similar tests verified that patient model and metrics plugin overrides are stored on the session when defined by the case. Invalid evaluator, patient model, or

metrics plugin configurations were tested to ensure session creation fails with the appropriate error responses.

Session-level access and state validation were further verified through audio submission tests. These confirmed that only the owning user may interact with a session, that completed sessions reject new turns, that unsupported audio formats fail validation, and that successful audio submissions return transcript data, patient response data, assistant audio references, and updated SPIKES-stage context.

8.2.2 Validation of Session

Validation assessed whether sessions provide a reliable execution context for structured clinical communication training. The tests show that a session consistently binds together a trainee, a case, and the runtime configuration needed to execute dialogue and evaluation logic. Reuse of active sessions supports continuity during ongoing practice, while forced creation of new sessions supports restarting or running separate attempts without ambiguity.

The validation results also show that sessions preserve reproducibility by freezing evaluator, patient model, and metrics selections at creation time rather than depending on mutable global settings during later execution. This supports consistent scoring, dialogue generation, and experiment repeatability across training runs.

Session validation further demonstrated that lifecycle and ownership constraints align with expected platform behavior. Only authorized users can interact with a session, completed sessions cannot accept additional turns, and audio-based interactions continue to operate even when optional analysis steps are unavailable. These results demonstrate that the session component provides a stable, secure, and reproducible container for end-to-end training interactions.

8.3 Case Management

The case management component defines structured clinical training scenarios used during session initialization and dialogue execution. Each case includes metadata such as script, objectives, difficulty, category, patient background, expected SPIKES flow, and optional plugin overrides. The component supports CRUD operations and filtering.

8.3.1 Verification of Case Management

Verification focused on correct CRUD behaviour, metadata persistence, and session integration. Unit tests validated case creation, retrieval, update, deletion, and listing functionality. Default values, including difficulty level, were verified during creation.

Metadata persistence was tested by creating cases with objectives, categories, patient backgrounds, expected SPIKES flow, and metrics plugins, then retrieving and comparing stored values. Filtering behaviour was verified by querying cases across different categories and difficulty levels.

Error handling was validated by requesting non-existent case IDs and confirming appropriate failures. Integration tests confirmed that case-level plugin overrides propagate correctly into sessions. All case-related tests passed successfully, confirming correct behaviour.

8.3.2 Validation of Case Management

Validation assessed whether cases support structured clinical communication scenarios and integrate correctly with downstream services. Cases containing scripts, objectives, and expected SPIKES flows were used to initialize sessions, confirming consistent scenario setup.

Case-level plugin configuration was validated to influence runtime behaviour, enabling different evaluators, patient models, and metrics without modifying core logic. Filtering and categorization were also validated to support progressive training workflows.

These results demonstrate that case management supports structured scenario definition, configurable training behaviour, and reproducible experimentation.

8.4 Dialogue

The dialogue component manages turn-by-turn interaction within a training session. It is responsible for analyzing clinician input, updating conversational context such as SPIKES stage, generating patient responses through the configured patient model, and optionally producing assistant audio for playback. The component also supports turn-level linguistic analysis, empathy-related signal detection, and secure retrieval of generated assistant audio.

8.4.1 Verification of Dialogue

Verification focused on correct turn processing, dialogue analysis, response generation, and assistant audio handling. Unit tests validated question classification for open and closed questions, detection of empathy cues, and intent analysis using the rule-based NLU layer. Additional turn-analysis tests verified that empathy-response metrics depend on detected response spans rather than broad emotional keywords alone, ensuring that scoring reflects actual communicative behavior.

Service-level verification confirmed that DialogueService uses the configured patient model plugin to generate patient replies and returns the plugin-produced text in the turn response. Tests also verified that detected conversational state is propagated correctly, including SPIKES-stage updates before patient response generation.

Optional text-to-speech behavior was also verified. When TTS is enabled, tests confirmed that assistant audio is synthesized, stored, and returned with an audio URL and expiration timestamp. Default behavior without TTS was verified to leave audio generation disabled, and failure handling was verified to ensure that dialogue still succeeds even if TTS generation fails. Voice selection inputs were also checked to confirm that patient characteristics influence synthesis instructions.

Dialogue-related API tests further verified assistant audio retrieval behavior. Tests confirmed that cached audio is served without unnecessary storage access, missing audio resources return appropriate not-found failures, expired audio is rejected, and access control restricts playback to the owning trainee or an administrator.

8.4.2 Validation of Dialogue

Validation assessed whether the dialogue component supports realistic, structured clinical communication practice. The tests show that clinician inputs can be analyzed for question type, empathy, and response characteristics before being passed into the dialogue pipeline, enabling meaningful downstream interpretation of conversational quality.

The service tests also validate that runtime dialogue behavior is configurable and scenario-aware. Patient responses are generated through the selected patient model rather than hardcoded logic, and updated SPIKES-stage context is propagated into response generation, supporting more coherent simulated patient behavior across different phases of a difficult conversation.

Optional assistant audio generation and retrieval were also validated as part of the interactive experience. The system can enrich patient replies with synthesized audio when enabled, while preserving usability when audio synthesis is unavailable. Secure access to generated assistant audio further supports realistic replay without exposing session content across users.

These results demonstrate that the dialogue component supports context-aware conversational exchange, configurable patient simulation, interpretable turn analysis, and optional multimodal interaction in a secure and resilient manner.

8.5 Audio

The audio component supports multimodal session interaction by handling clinician voice capture, speech-to-text transcription, optional tone analysis, assistant speech synthesis, storage and caching of generated audio, cleanup of expired audio artifacts, and secure playback of assistant responses. It bridges browser recording capabilities with backend processing and retrieval services.

8.5.1 Verification of Audio

Verification focused on correct recording, transcription, synthesis, caching, cleanup, and playback behavior. Backend adapter tests validated that the Whisper-based transcription layer rejects empty audio input, trims returned transcripts, rejects blank transcription results, and maps provider failures into application-level service errors.

Audio submission tests verified that supported uploaded audio can be transcribed successfully and returned with transcript data and optional tone metadata. These tests also confirmed that invalid file formats are rejected before transcription, unauthorized users cannot submit audio to another user’s session, completed sessions reject new

audio turns, and transcription failures are surfaced correctly. Best-effort behavior was verified by confirming that optional tone analysis and storage failures do not block transcript processing when core input handling can still proceed.

Assistant speech generation was also verified within dialogue processing. When text-to-speech is enabled, tests confirmed that synthesized assistant audio is created, stored, and returned with an audio URL and expiration timestamp. Complementary tests verified that TTS remains disabled by default and that dialogue continues successfully if synthesis fails. Voice-profile tests further verified that case metadata influences voice selection and synthesis instructions for different patient demographics and speaking styles.

Caching and playback behavior were independently verified. Cache service tests confirmed that stored audio files can be retrieved locally and that least-recently-used eviction occurs when cache size limits are exceeded. Cleanup tests verified that expired assistant audio is deleted from storage and cache, while fresh assistant audio and user-uploaded audio remain untouched. Additional API tests confirmed that assistant audio playback uses cache when available, fetches from storage on cache miss, rejects expired or missing audio, and enforces access control so only the owning trainee or an administrator can retrieve the file. Frontend tests verified MIME-type selection, recording file extension mapping, browser recording capability detection, voice-activity-based upload gating, blob fetching for playback, object URL creation, audio playback lifecycle handling, and proper teardown of object URLs after playback.

8.5.2 Validation of Audio

Validation assessed whether the audio subsystem supports realistic spoken interaction while remaining robust and secure. The tests show that spoken clinician input can be captured in the browser, screened for meaningful speech activity, transmitted in supported formats, and converted into usable transcript text for downstream dialogue handling.

The results also validate that audio-enhanced patient interaction is practical rather than fragile. Optional tone analysis can enrich the interaction when available, while synthesis and storage failures do not prevent core conversational progress. Case-aware voice profile selection further supports more believable assistant speech output for different simulated patient scenarios.

Audio lifecycle management was also validated end to end. Generated assistant audio can be cached for efficient replay, served securely to authorized users, and cleaned up once expired without affecting active or user-owned artifacts. These results demonstrate that the audio component provides reliable speech input, resilient audio output, efficient playback, and controlled retention for multimodal clinical training sessions.

8.4 Scoring Service and Feedback

The scoring and feedback component evaluates completed clinical conversations by producing quantitative performance metrics and qualitative insights. The scoring service computes metrics such as empathy, communication, SPIKES completion, and overall score based on detected empathic opportunities, response alignment, and stage coverage. The feedback layer then translates these metrics into user-facing outputs, including strengths, areas for improvement, and narrative feedback. In this pipeline, scoring represents quantitative evaluation, while feedback represents qualitative interpretation.

8.4.1 Verification of Scoring Service

Verification focused on ensuring correct and reliable execution of the scoring pipeline. Scoring is triggered only upon session closure and uses a correctly ordered transcript with associated annotations. The system was tested to handle both valid and invalid evaluator outputs, enforce score bounds, and generate exactly one feedback record per completed session. Failure cases were validated to ensure explicit error handling rather than partial outputs. Performance testing confirmed that scoring completes within an acceptable time frame, supporting real-time feedback delivery.

8.4.2 Validation of Scoring System

Validation assessed whether the scoring outputs reflect clinical communication quality. Four standardized scenarios (strong, decent, mixed, weak) were constructed using an annotated breaking bad news dataset provided by Dr. Lahnala, and were reviewed and approved by her to ensure clinical and educational validity. Each scenario was evaluated using the baseline, hybrid v1, and hybrid v2 evaluators.

To approximate ground truth, a multi-model external judge panel independently scored each scenario using a consistent schema. The panel outputs were also reviewed and approved by the supervisor. For each metric, the mean and standard deviation across models were computed to represent consensus and agreement. This approach aggregates independent evaluations to reduce individual model bias and improve the stability of the estimated ground truth.

Case	Metric	Panel (Mean ± Std)	Baseline	Hybrid v1	Hybrid v2
Strong	Empathy	92.0 ± 3.74	75.48	90.0	90.0
	Communication	92.75 ± 2.77	74.25	85.0	95.0
	SPIKES Completion	96.75 ± 3.42	100.0	100.0	100.0
	Overall	93.5 ± 2.69	82.59	90.5	94.0
Decent	Empathy	71.5 ± 10.59	85.0	90.0	85.0
	Communication	68.5 ± 11.28	69.25	85.0	85.0

	SPIKES Completion	60.18 ± 11.65	83.33	83.3	83.3
	Overall	66.25 ± 11.23	81.35	87.3	84.49
Mixed	Empathy	24.5 ± 4.27	20.0	40.0	30.0
	Communication	44.25 ± 8.17	67.31	70.0	75.0
	SPIKES Completion	43.58 ± 15.77	66.67	66.7	83.3
	Overall	35.5 ± 6.69	33.46	52.5	54.99
Weak	Empathy	10.75 ± 4.38	5.0	20.0	20.0
	Communication	30.5 ± 5.32	55.46	60.0	70.0
	SPIKES Completion	30.25 ± 12.77	16.67	50.0	60.0
	Overall	21.5 ± 3.77	16.09	35.0	42.0

The panel demonstrated clear separation between performance levels, with strong cases scoring high (overall mean = 93.5 ± 2.69) and weak cases scoring low (21.5 ± 3.77). Model outputs were compared to panel means using mean absolute error (MAE). Hybrid v1 showed the closest alignment among LLM-based evaluators (≈15.53), followed by hybrid v2 (≈17.11), while the baseline achieved ≈14.32 but is not directly comparable due to its rule-based design.

Acceptance criteria for scoring validation were defined as: preservation of performance ordering (strong > decent > mixed > weak), monotonic decrease in scores across scenario difficulty, mean absolute error within an acceptable range (≤ 20), and SPIKES completion reflecting expected stage coverage. All evaluators satisfied directional consistency and preserved ordering across performance levels, indicating acceptable scoring reliability.

All evaluators exhibited a tendency to overestimate performance in lower-quality scenarios, particularly in communication and SPIKES scores. Despite this, the results demonstrate that the scoring system produces directionally consistent evaluations aligned with multi-model consensus.

It is important to note that the scores presented in this validation correspond to the baseline (rule-based) and raw LLM evaluator outputs. The system also includes a merged scoring layer, currently implemented as a weighted combination of rule-based and LLM scores (70% rule, 30% LLM), which is displayed in the user-facing feedback interface. This merged layer is intentionally designed as a tunable aggregation mechanism to support future experimentation and model calibration. This design direction was guided by the project supervisor, Dr. Lahnala, to enable flexibility for ongoing research and iterative refinement of scoring strategies. As such, the merged scores are not included in the validation analysis, and no claims are made regarding their alignment with external

benchmarks. Instead, validation focuses on the underlying scoring components, which provide a stable and interpretable foundation for future development.

8.7 Plugin System Verification and Validation

8.7.1 Verification of Plugin System

Verification focused on the correct and reliable execution of the plugin-based architecture across patient simulation, evaluation, and metrics components. The plugin manager was tested to resolve and instantiate plugins using dynamic class paths. Tests confirmed that valid plugins load successfully and that invalid configurations produce explicit errors. Interface compliance was verified to ensure plugins conform to the PatientModel, Evaluator, and MetricsPlugin contracts.

Execution correctness was validated through unit and integration tests covering patient model response generation, evaluator output structure and score bounds, metrics plugin aggregation, and plugin registry behaviour. Multi-plugin execution was tested to ensure multiple metrics plugins run without overwriting outputs. Performance testing confirmed that scoring completes within acceptable time bounds. All plugin tests passed successfully, confirming system stability.

8.7.2 Validation of Plugin Architecture

Validation assessed whether the plugin architecture supports extensible and research-oriented experimentation. Tests confirmed that different patient models, evaluators, and metrics plugins can be configured and executed without modifying core logic. Case-level and session-level plugin configurations were validated to propagate correctly during runtime.

Validation also confirmed that multiple metrics plugins can execute concurrently and produce aggregated outputs, supporting comparative analytics. Session-level storage of plugin configuration ensures reproducibility across experiments. These results demonstrate that the plugin architecture enables modular experimentation and flexible evaluation strategies for clinical communication training.

8.8 Database

8.8.1 Verification of Database

Verification covered schema wiring, type behaviour, entity constraints, migration integrity, and transactional persistence across the primary simulation tables. Automated validation from the tests/persistence package produced 19 passing tests in a standard run, providing repeatable evidence of database correctness.

Test coverage included session, turn, and feedback create/read/update operations; JSON persistence for session metadata and turn annotations; enforcement of a single feedback row per session via a unique constraint; and proper foreign-key rejection

following prior successful commits. Duplicate-feedback scenarios were also validated, confirming that the original record remains unchanged.

The UTCDateTimeType was verified across multiple conditions, including aware UTC timestamps, naive-as-UTC handling, offset normalization, NULL values, and correct bind shaping for storage. User and Case entity constraints were validated, including unique email enforcement and required core fields. Additionally, the system correctly allows multiple open sessions per user/case where product requirements permit.

Migration integrity was confirmed through Alembic verification, ensuring a single head and consistent down_revision chains. Optional PostgreSQL runs repeated core persistence checks and properly cleaned up test artifacts using a test-only database URL.

Together, these results provide repeatable, quantitative validation of the database layer, with 19/19 required checks passing and 2/2 additional checks passing under PostgreSQL, supporting the reliability and integrity of the simulation data infrastructure described in the project deliverables.

8.8.2 Validation of Database

Validation assesses whether the collected evidence supports the intended product behavior: durable simulation state, a single final scored outcome per session enforced at the database level, stable partial-failure semantics when late writes fail, consistent UTC-based timestamps for auditing and APIs, and strong identity and case integrity for users and scenarios. It also confirms a linear, reviewable migration history to support reliable releases.

This validation is bounded by the testing environment. Routine verification is performed using SQLite with explicit foreign-key enforcement, while production behavior depends on Alembic-managed PostgreSQL DDL.

8.9 Research

The research component provides administrators with anonymized session and transcript data for downstream analysis. It covers PII redaction across all export paths, deterministic pseudonymous identifier generation, and generator-based CSV streaming for large datasets.

8.9.1 Verification of Research

Verification confirmed correct and reliable behavior of the research pipeline. The generate_anon_session_id function was verified to be deterministic, to never embed the raw integer session ID in its output, and to produce unique identifiers across distinct sessions. The resolve_anon_to_session_id helper was verified to return the correct integer for a valid anon ID and None for unknown inputs. The _anonymize_text function was verified to redact email addresses, phone numbers, personal names, and long numeric sequences. The _anonymize_session mapper was verified to handle sessions

with and without turns or feedback. The get_all_sessions method was verified to return flattened rows in which clinical_score mirrors overall_score, with a fallback when communication_score is absent. Export methods were verified to produce correctly structured JSON arrays and CSV files with the expected headers and row counts. The GET /v1/research/export/session/{anon_id}.csv endpoint was verified to return 404 for unknown anon IDs before the streaming response begins, preventing a mid-stream failure. Non-admin requests to all research endpoints were verified to be rejected with 403. The frontend API client was verified to issue correctly formed requests, convert 403 responses into user-facing access-denied errors, and propagate other errors unchanged.

8.9.2 Validation of Research

Validation assessed whether the research exports are free of personally identifiable information. A session was seeded with turns containing six known PII patterns: an email address, a phone number in two formats, two personal names, and a ten-digit numeric identifier. All export paths were scanned against these patterns: the JSON export, full CSV export, metrics CSV stream, transcripts CSV stream, session detail endpoint, and individual session transcript CSV stream. No PII was detected in any path.

Additional anonymization properties were validated: anon IDs are stable across repeated exports of the same session with the same salt; changing the salt produces different anon IDs; twenty distinct sessions all receive distinct anon IDs with a fixed salt, confirming no collision. Exported scores in the JSON and metrics CSV were confirmed to match the corresponding stored feedback records exactly, and the session count in the JSON export was confirmed to match the database count up to the export limit.

Performance was validated against a dataset of 100 sessions and 250 turns. Streaming the metrics CSV completed in under 5 seconds; streaming the transcripts CSV completed in under 5 seconds; worst-case anon ID resolution by linear scan completed in under 5 seconds. Streaming delivery was confirmed to be incremental: both stream methods yield multiple chunks rather than a single in-memory blob. Ten concurrent get_all_sessions calls were confirmed to return consistent results, validating read safety under concurrent access.

8.10 Analytics

The analytics component provides two views of session performance data: a per-session breakdown for individual trainees and an aggregated dashboard for administrators. The trainee view maps scored feedback fields from the database into a structured model. The admin dashboard aggregates user, session, performance, and case statistics across the full dataset.

8.10.1 Verification of Analytics

Verification confirmed correct behavior of both analytics pipelines. The admin get_dashboard_analytics method was verified to return a valid AnalyticsDashboard

containing all four sub-stat blocks. User counts, role breakdowns, and active-user counts were verified to map correctly from the repository layer. Completed and active session counts were verified to derive correctly from the state dictionary, including defaulting to zero when keys are absent. Score keys were verified to map to the correct fields, and monthly averages were verified to be wrapped in structured objects. Case counts and category breakdowns were verified to pass through unchanged. All-zero and empty input was verified to produce a structurally valid response without error.

For the trainee service, the _parse_json helper was verified to handle None, pre-parsed dictionaries, valid JSON strings, and malformed strings without raising. The get_user_session_analytics method was verified to return only the requesting user's completed sessions in descending date order. SPIKES coverage percent and empathy opportunity addressed rate were verified to derive correctly from stored JSON fields. Sessions with null scores or missing feedback were verified to produce zero-valued fields rather than errors.

The GET /v1/analytics/my-sessions endpoint was verified to return 200 with the correct response shape for an authenticated trainee, to exclude active sessions, to isolate each user's data so sessions belonging to other users do not appear, and to reject unauthenticated requests with 403. The response schema was verified to include all required fields. The frontend API client was verified to issue the correct request, map snake_case DTO fields to camelCase model fields, and propagate errors unchanged.

8.10.2 Validation of Analytics

Validation assessed whether the analytics outputs are accurate and stable under realistic data conditions. Four robustness scenarios were constructed: sessions with fully null feedback scores, sessions with malformed SPIKES coverage JSON, research exports for sessions without feedback, and research exports for sessions referencing a non-existent case. In each case the service returned a valid result without raising, with null or zero values substituted as appropriate.

Performance was validated against a dataset of 100 completed sessions across 5 users and 3 cases. Querying analytics for a single user (approximately 20 sessions) completed in under 3 seconds. Querying all five users sequentially completed in under 5 seconds. The admin dashboard pipeline, tested with mocked repositories representing 1000 users and 5000 sessions, completed in under 1 second.

8.11 Admin

8.11.1 Verification of Admin

Verification confirms that the admin subsystem behaves as expected, based on 20 automated tests in the dedicated admin test suite, all of which passed in recorded runs. The coverage includes both integration tests (admin workflows using an isolated database) and focused unit/repository tests (such as dashboard aggregation with mocked data and user overview queries).

The tests also verify access control, ensuring that non-admin and unauthenticated users are properly restricted. Invalid inputs and missing resources return the correct error responses. A mocked performance check provides a simple timing measurement for dashboard assembly, and empty-data scenarios are tested to confirm aggregation logic behaves correctly.

Overall, this evidence provides repeatable and deterministic validation of key admin functionality, including analytics dashboards, user rollups, session review (with feedback and turn-level metrics), plugin visibility, database readiness signaling, and admin-only case creation.

8.11.2 Validation of Admin

Validation focused on whether these behaviors support the intended administrator workflows: reliable cohort summaries, useful user-level rollups, clear session reviews with feedback and turn-level metrics, and visibility into which plugins are active versus simply available.

The validation is based on alignment with documented admin capabilities, expected UI usage patterns, and access-control requirements, along with consistent pass results from the verification tests.

Any remaining risk is limited to scenarios not covered by the automated tests, such as production-scale data volumes or live authentication flows. These cases can be addressed through operational testing or user-acceptance validation if required by the program.

8.12 Summary

The validation and verification activities described above provide confidence that system components behave correctly, meet performance expectations, and degrade safely under failure conditions. Quantitative metric values and test results will be reported in the final submission.

9.0 Appendix A

9.1 Database

The application uses a PostgreSQL database to store information related to users, synthetic patient cases, dialogue sessions, turn-level conversation logs, and generated feedback. The core tables and their DDL are provided below:

9.1.1 Cases Table

Column Name	Data Type	Description	Constraint

id	INTEGER	A unique identifier for each patient case	PRIMARY KEY
title	VARCHAR	The title of the patient case	NOT NULL
description	TEXT	A brief description of the case scenario	NOT NULL
script	TEXT	The base script or narrative used to guide the simulated patient responses	NOT NULL
objectives	TEXT	Learning objectives associated with the case	NOT NULL
difficulty_level	VARCHAR	The difficulty level of the case (e.g., easy, medium, hard)	NOT NULL
category	VARCHAR	The category or type of patient scenario	NOT NULL
patient_background	TEXT	Background information about the synthetic patient	NOT NULL
expected_spikes_flow	TEXT	The expected progression of the case aligned with SPIKES stages	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the case was created	NOT NULL
updated_at	TIMESTAMP	Timestamp indicating when the case was last updated	NOT NULL

9.1.2 Users Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each user	PRIMARY KEY

email	VARCHAR	The email address associated with the user account	UNIQUE, NOT NULL
hashed_password	VARCHAR	Securely hashed password for authentication	NOT NULL
role	VARCHAR	Role of the user (e.g., trainee, admin)	NOT NULL
full_name	VARCHAR	Full name of the user	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the user account was created	NOT NULL
updated_at	TIMESTAMP	Timestamp indicating when the user account was last updated	NOT NULL

9.1.3 Turns Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each conversation turn	PRIMARY KEY
session_id	INTEGER	Identifier of the conversation session this turn belongs to	FOREIGN KEY
user_id	INTEGER	Identifier of the user who produced the turn	FOREIGN KEY
turn_number	INTEGER	Sequential order of the turn within a session	NOT NULL
role	VARCHAR	Role of the speaker (e.g., learner, patient)	NOT NULL
text	TEXT	Text content of the turn	NOT NULL
audio_url	VARCHAR	Optional URL linking to the audio recording of the turn	NULL

metrics_json	TEXT	Serialized metrics related to empathy and communication analysis	NULL
spans_json	TEXT	Serialized span-level annotations for the turn	NULL
relations_json	TEXT	Serialized relationships between annotated spans	NULL
spikes_stage	VARCHAR	SPIKES stage associated with this turn	NULL
timestamp	TIMESTAMP	Timestamp indicating when the turn occurred	NOT NULL

9.1.4 Sessions Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each conversation session	PRIMARY KEY
user_id	INTEGER	Identifier of the user who initiated the session	FOREIGN KEY
case_id	INTEGER	Identifier of the patient case used in the session	FOREIGN KEY
state	VARCHAR	Current lifecycle state of the session (e.g., active, completed)	NOT NULL
current_spikes_stage	VARCHAR	Current SPIKES stage of the session	NULL
started_at	TIMESTAMP	Timestamp indicating when the session started	NOT NULL
ended_at	TIMESTAMP	Timestamp indicating when the session ended	NULL
duration_seconds	INTEGER	Total duration of the session in seconds	NULL

metadata	TEXT	Additional session-level metadata stored in serialized form	NULL

9.1.5 Feedback Table:

Column Name	Data Type	Description	Constraint
id	INTEGER	A unique identifier for each feedback record	PRIMARY KEY
session_id	INTEGER	Identifier of the session this feedback corresponds to	FOREIGN KEY
empathy_score	DOUBLE PRECISION	Quantitative score representing empathetic communication	NOT NULL
communication_score	DOUBLE PRECISION	Score reflecting overall communication effectiveness	NOT NULL
spikes_completion_score	DOUBLE PRECISION	Score indicating completion of SPIKES stages	NOT NULL
overall_score	DOUBLE PRECISION	Aggregate performance score for the session	NOT NULL
eo_counts_by_dimension	TEXT	Counts of empathic opportunities grouped by empathy dimension	NULL
elicitation_counts_by_type	TEXT	Counts of elicitation strategies used by type	NULL
response_counts_by_type	TEXT	Counts of empathic responses grouped by response type	NULL
eo_counts	TEXT	Total counts of empathic opportunities identified	NULL

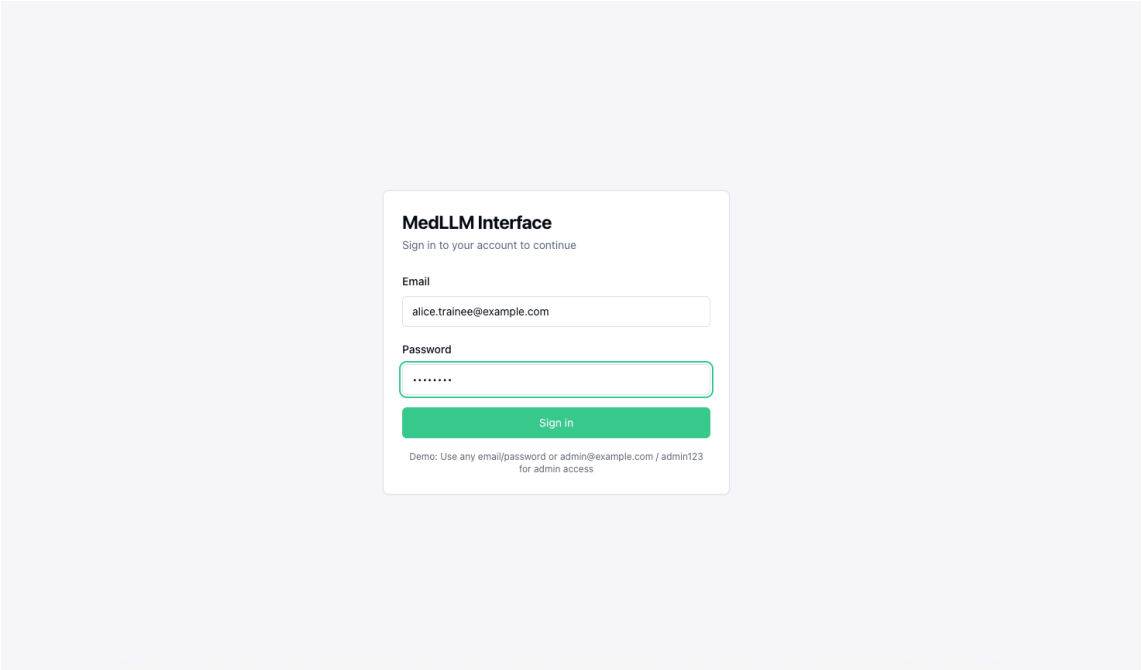
elicitation_count	INTEGER	Total number of elicitation attempts	NULL
empathy_response_count	INTEGER	Total number of empathic responses	NULL
linkage_stats	TEXT	Statistics describing linkages between EO, elicitation, and responses	NULL
missed_opportunities	TEXT	Identified missed empathic opportunities	NULL
missed_opportunities_by_dimension	TEXT	Missed opportunities grouped by empathy dimension	NULL
eo_to_elicitation_links	TEXT	Mappings between empathic opportunities and elicitation attempts	NULL
eo_to_response_links	TEXT	Mappings between empathic opportunities and responses	NULL
response_types	TEXT	Distribution of response types used by the trainee	NULL
spikes_coverage	TEXT	Coverage information for SPIKES stages	NULL
spikes_timestamps	TEXT	Timestamps associated with SPIKES stage transitions	NULL
spikes_strategies	TEXT	Strategies used by the trainee for each SPIKES stage	NULL

question_breakdown	TEXT	Breakdown of question types used during the session	NULL
bias_probe_info	TEXT	Metadata related to bias probe analysis, if applicable	NULL
evaluator_meta	TEXT	Metadata about the evaluation process or model configuration	NULL
latency_ms_avg	DOUBLE PRECISION	Average model response latency in milliseconds	NULL
strengths	TEXT	Identified communication strengths	NOT NULL
areas_for_improvement	TEXT	Suggested areas for improvement	NOT NULL
detailed_feedback	TEXT	Detailed narrative feedback for the session	NOT NULL
created_at	TIMESTAMP	Timestamp indicating when the feedback was generated	NOT NULL

9.2 User Interface Flow and Screenshots

9.2.1 Login and Authentication Screen

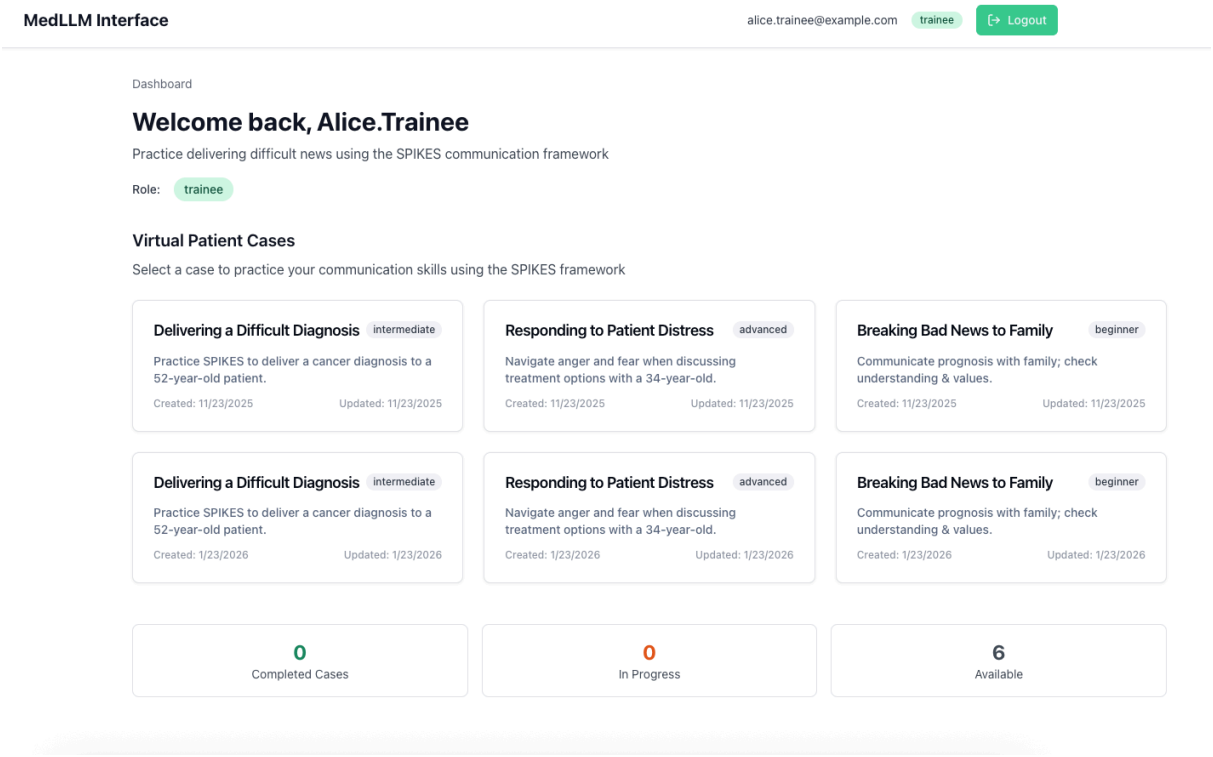
Figure A.1 shows the login interface used to authenticate users before accessing the APEX system. Valid credentials are needed to proceed to the case selection dashboard. Authentication is implemented and enforced by the backend API. If invalid credentials are provided, they are rejected with an error message



9.2.2 Case Selection Dashboard

Figure A.2 shows the case selection dashboard, where users choose from predefined “Virtual Patient Cases”. A user selects a case by clicking a case tile and is redirected to the chat interface for that case, initializing a new training session. Each case displays

relevant metadata such as title, difficulty level, created/updated dates and learning objectives.



9.2.3 Chat-Based Training Interface

Figure A.3a shows the clinical case script and contextual information presented to the LLM for the active training session. The view defines the case metadata, and the script defines the virtual patient's background, emotional state, clinical context, and

behavioural rules. As well, following the SPIKES format, the expected stages are outlined for this scenario. The script is used to create a flow for speech and conversation with the virtual simulated patient. This primarily ensures that responses from the dialogue remain consistent with the intended learning objectives.

Delivering a Difficult Diagnosis

Practice SPIKES to deliver a cancer diagnosis to a 52-year-old patient.

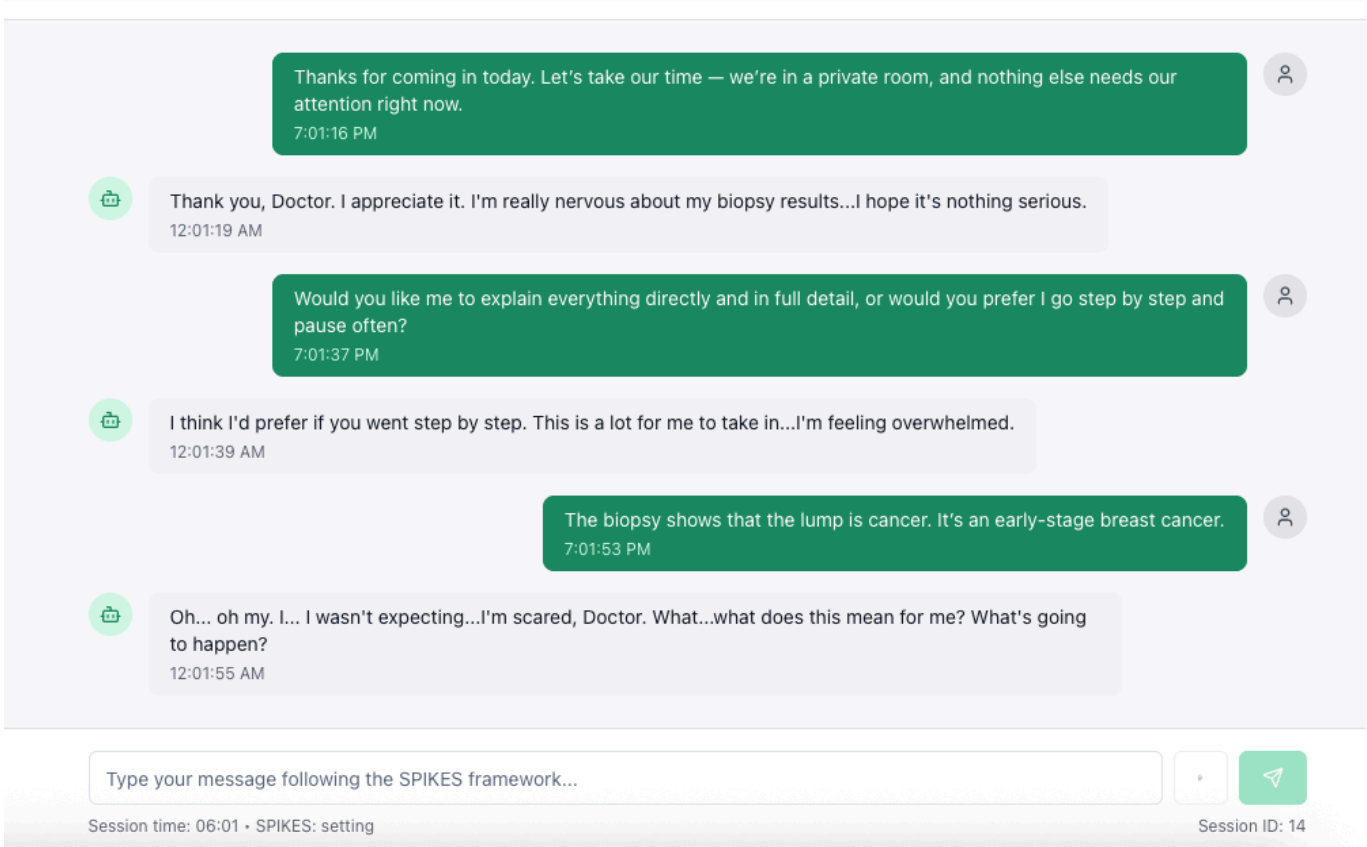
End Session

Session Timer	Difficulty	Category	SPIKES Stage
04:25	intermediate	Breaking Bad News	Setting

Script

```
[Persona]

You are a 52-year-old woman who recently had a breast biopsy. You value honesty but are very anxious about cancer. You ask follow-up questions and prefer a calm environment so you
can absorb difficult news.


[ClinicalContext]

You know a biopsy was taken for a suspicious lump and have been told results are pending. Today the clinician will disclose that it is early-stage cancer and explain the next steps.


[SPIKES]

Setting: You appreciate a private, calm room with the clinician sitting close.

Perception: You say you have tried not to worry but thoughts about the diagnosis dominate your mind.

Invitation: You want full transparency and need the clinician to speak directly with you.

Knowledge: The word cancer makes you emotional, so clear non-technical language helps you stay present.

Emotions: You may well up and need pauses; empathy keeps you connected.

Strategy: You need a concrete plan with follow-up and support resources.


[BehaviorRules]

- If the clinician avoids naming 'cancer', you ask directly, "Is it cancer?"
- If the clinician uses jargon, you ask, "Can you explain that more simply?"
- If the clinician shows empathy, you calm down and respond more openly.
```

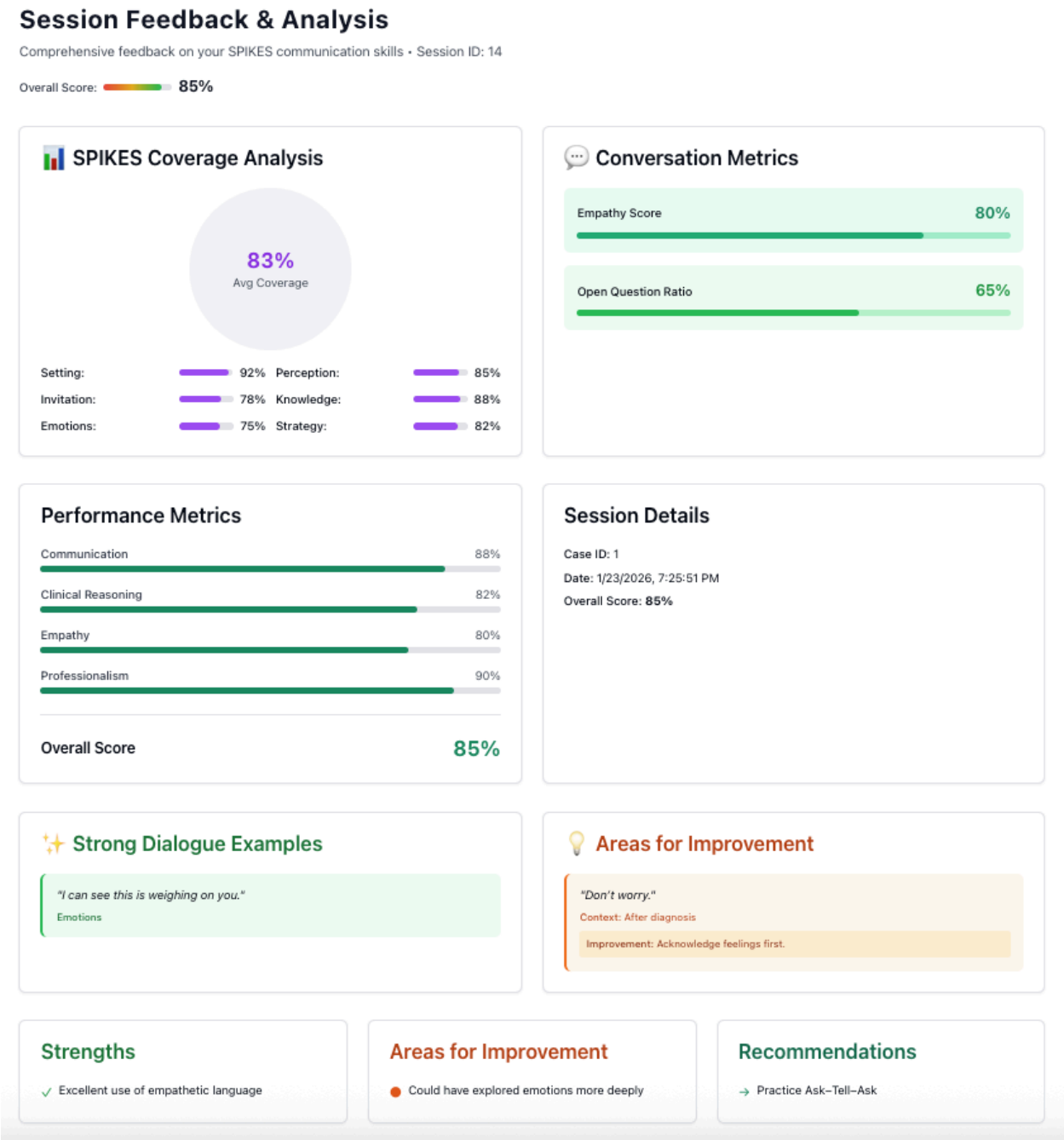
Figure A.3b shows the chat conversation with the virtual patient. The chat input is used to accept dialogue that is then processed by the backend and the external LLM provider. The virtual patient's response is then displayed in the chat.



9.2.4 Session Feedback and Scoring View

Figure A.4 shows the feedback and analysis screen after ending the conversation with the virtual patients. SPIKES coverage analysis provides percentage metrics for each

SPIKES component reached. Some additional details regarding the conversation are provided, such as: strengths, areas for improvement, recommendations, strong dialogue examples and conversation metrics.



9.3

Figure A.5, Project Component Diagram

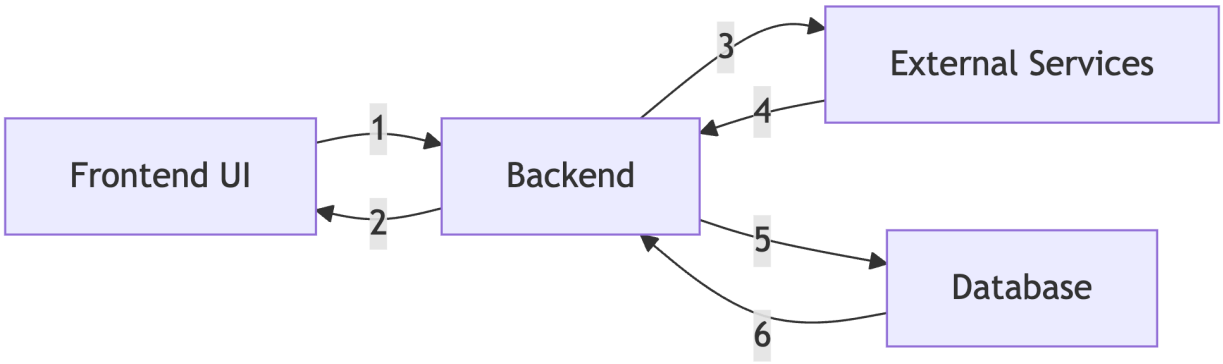
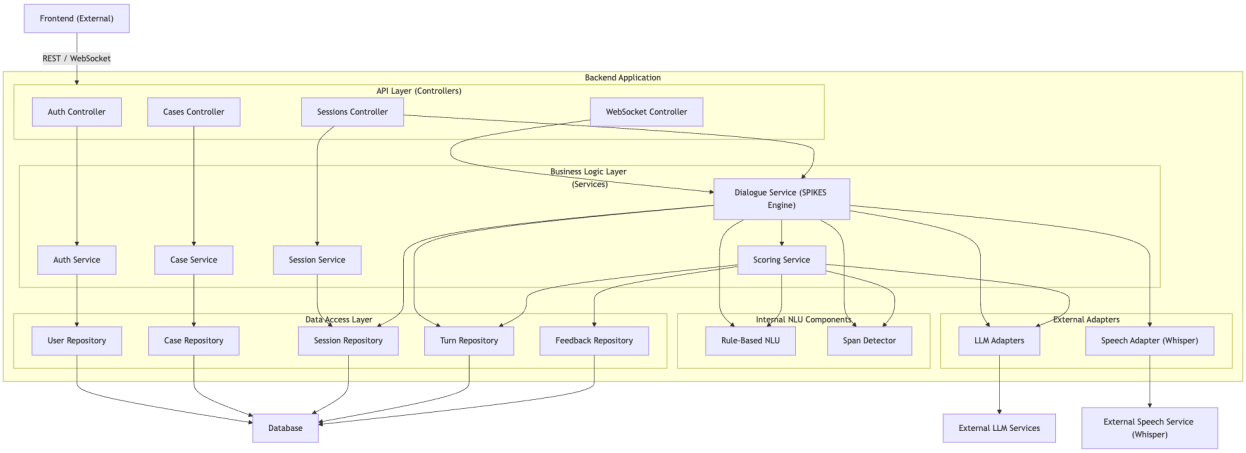



Figure A.6 Backend Component Diagram